Лабораторная работа №9

Тема работы: «Разработка алгоритмов и программ с использованием методов внутренней сортировки, методов поиска информации. Проведение сравнительного анализа различных алгоритмов сортировки»

1 Цель работы

Научить разрабатывать алгоритмы и программы с использованием методов внутренней сортировки. Сформировать умение анализировать эффективность различных методов сортировки массивов

2 Задание

Часть 1.

Разработать программу с использованием трех функций: «заполнение массива», «сортировка массива», «вывод массива на экран» Создать массив заданного количества строк. Каждая строка содержит фамилию и имя учащегося. Необходимо отсортировать элементы массива по фамилии учащихся в алфавитном порядке, учитывая регистр.

```
#include <iostream>
#include <string>
#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <conio.h>

using namespace std;

const int SIZE = 8;

void task1();
void task2();
void sortAndPrintArray(int choice);

int main() {
    setlocale(LC_ALL, "RUS");
    char choice;

    while (true) {
        cout << "\nУСЛОВИЯ ЗАДАНИЙ\n";
        cout << "1 - Сортировка элементов массива по фамилии учащихся в алфавитном порядке
(учёт регистра)\n";
        cout << "2 - Реализация сортировок целочисленного массива\n";
        cout << "0 - Выход\n";
        cout << "Выберите номер задания: ";
        cin >> choice;

        switch (choice) {
        case '1':
            task1();
            break;
        case '2':
            task2();
            break;
        case '0':
            return 0;
```

```
        default:
            cout << "Неверный выбор\n";
            break;
        }

        cout << "\nНажмите ESC для выхода или любую другую клавишу для продолжения...\n";
        char key = _getch();
        if (key == 27) break;
    }

    return 0;
}



void task1() {
    int count;
    cout << "Введите количество учащихся: ";
    cin >> count;

    if (count <= 0) {
        cout << "Некорректное количество." << endl;
        return;
    }

    string* students = new string[count];
    cin.ignore();

    for (int i = 0; i < count; ++i) {
        cout << "Введите фамилию и имя учащегося №" << i + 1 << ": ";
        getline(cin, students[i]);
    }

    for (int i = 0; i < count - 1; ++i) {
        for (int j = i + 1; j < count; ++j) {
            string lastNameI = students[i].substr(0, students[i].find(' '));
            string lastNameJ = students[j].substr(0, students[j].find(' '));
            if (lastNameJ < lastNameI) {
                swap(students[i], students[j]);
            }
        }
    }

    cout << "\nОтсортированный список учащихся:\n";
    for (int i = 0; i < count; ++i) {
        cout << students[i] << endl;
    }

    delete[] students;
}
```

```
УСЛОВИЯ ЗАДАНИЙ
1 - Сортировка элементов массива по фамилии учащихся в алфавитном порядке (учёт регистра)
2 - Реализация сортировок целочисленного массива
0 - Выход
Выберите номер задания: 1
Введите количество учащихся: 5
Введите фамилию и имя учащегося №1: Krivniuk Diana
Введите фамилию и имя учащегося №2: Ramluk Natasha
Введите фамилию и имя учащегося №3: Bexter Polina
Введите фамилию и имя учащегося №4: Malec Arina
Введите фамилию и имя учащегося №5: Podsosnyi Alexander

Отсортированный список учащихся:
Bexter Polina
Krivniuk Diana
Malec Arina
Podsosnyi Alexander
Ramluk Natasha
```

Рисунок 1 - Выполнение задачи №1

Часть 2.

Разработать программу с использованием конструкции switch-case для выбора варианта сортировки. Каждую итерацию сортировки выводить на экран. Задать рандомно целочисленный массив из 8 элементов. Отсортировать его в порядке возрастания с использованием сортировки Шелла, вставками/выбором, пузырьком.

```
void sortAndPrintArray(int choice) {
    srand(time(0));
    int original[SIZE];
    for (int i = 0; i < SIZE; ++i)
        original[i] = rand() % 100;

    cout << "Исходный массив:\n";
    for (int i = 0; i < SIZE; ++i)
        cout << original[i] << " ";
    cout << endl;

    int arr[SIZE];
    copy(original, original + SIZE, arr);

    switch (choice) {
    case 1:
        cout << "\nСортировка пузырьком:\n";
        for (int i = 0; i < SIZE - 1; ++i) {
            for (int j = 0; j < SIZE - i - 1; ++j)
                if (arr[j] > arr[j + 1])
                    swap(arr[j], arr[j + 1]);
            cout << "Шаг " << i + 1 << ": ";
            for (int k = 0; k < SIZE; ++k)
                cout << arr[k] << " ";
            cout << endl;
        }
        break;

    case 2:
        cout << "\nСортировка вставками:\n";
        for (int i = 1; i < SIZE; ++i) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                --j;
            }
            arr[j + 1] = key;
            cout << "Шаг " << i << ": ";
            for (int k = 0; k < SIZE; ++k)
                cout << arr[k] << " ";
            cout << endl;
        }
        break;

    case 3:
        cout << "\nСортировка выбором:\n";
        for (int i = 0; i < SIZE - 1; ++i) {
            int minIdx = i;
            for (int j = i + 1; j < SIZE; ++j)
                if (arr[j] < arr[minIdx])
                    minIdx = j;
            swap(arr[i], arr[minIdx]);
            cout << "Шаг " << i + 1 << ": ";
            for (int k = 0; k < SIZE; ++k)
                cout << arr[k] << " ";
            cout << endl;
```

```
        }
        break;

    case 4:
        cout << "\nСортировка Шелла:\n";
        for (int gap = SIZE / 2, step = 1; gap > 0; gap /= 2, ++step) {
            for (int i = gap; i < SIZE; ++i) {
                int temp = arr[i];
                int j;
                for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)
                    arr[j] = arr[j - gap];
                arr[j] = temp;
            }
            cout << "Шаг " << step <<":  ";
            for (int k = 0; k < SIZE; ++k)
                cout << arr[k] << " ";
            cout << endl;
        }
        break;

    default:
        cout << "Неверный выбор метода сортировки.\n";
        return;
    }

    cout << "\nОтсортированный массив:\n";
    for (int i = 0; i < SIZE; ++i)
        cout << arr[i] << " ";
    cout << endl;
}

void task2() {
    int choice;
    cout << "\nВыберите метод сортировки:\n";
    cout << "1 - Пузырьком\n2 - Вставками\n3 - Выбором\n4 - Шелла\nВаш выбор: ";
    cin >> choice;
    sortAndPrintArray(choice);
}
```

```
Выберите метод сортировки:
1 - Пузырьком
2 - Вставками
3 - Выбором
4 - Шелла
Ваш выбор: 1
Исходный массив:
36 65 53 33 6 22 56 11

Сортировка пузырьком:
Шаг 1: 36 53 33 6 22 56 11 65
Шаг 2: 36 33 6 22 53 11 56 65
Шаг 3: 33 6 22 36 11 53 56 65
Шаг 4: 6 22 33 11 36 53 56 65
Шаг 5: 6 22 11 33 36 53 56 65
Шаг 6: 6 11 22 33 36 53 56 65
Шаг 7: 6 11 22 33 36 53 56 65

Отсортированный массив:
6 11 22 33 36 53 56 65
```

Рисунок 2 - Пример выполнения задачи №2

```
Выберите метод сортировки:
1 - Пузырьком
2 - Вставками
3 - Выбором
4 - Шелла
Ваш выбор: 2
Исходный массив:
79 22 11 76 24 1 50 24

Сортировка вставками:
Шаг 1: 22 79 11 76 24 1 50 24
Шаг 2: 11 22 79 76 24 1 50 24
Шаг 3: 11 22 76 79 24 1 50 24
Шаг 4: 11 22 24 76 79 1 50 24
Шаг 5: 1 11 22 24 76 79 50 24
Шаг 6: 1 11 22 24 50 76 79 24
Шаг 7: 1 11 22 24 24 50 76 79

Отсортированный массив:
1 11 22 24 24 50 76 79
```

Рисунок 3 - Пример выполнения задачи №2

```
Выберите метод сортировки:
1 - Пузырьком
2 - Вставками
3 - Выбором
4 - Шелла
Ваш выбор: 3
Исходный массив:
10 99 50 64 54 5 13 86

Сортировка выбором:
Шаг 1: 5 99 50 64 54 10 13 86
Шаг 2: 5 10 50 64 54 99 13 86
Шаг 3: 5 10 13 64 54 99 50 86
Шаг 4: 5 10 13 50 54 99 64 86
Шаг 5: 5 10 13 50 54 99 64 86
Шаг 6: 5 10 13 50 54 64 99 86
Шаг 7: 5 10 13 50 54 64 86 99

Отсортированный массив:
5 10 13 50 54 64 86 99
```

Рисунок 4 - Пример выполнения задачи №2

```
Исходный массив:
45 52 50 18 40 17 33 30

Сортировка Шелла:
Шаг 1:  40 17 33 18 45 52 50 30
Шаг 2:  33 17 40 18 45 30 50 52
Шаг 3:  17 18 30 33 40 45 50 52

Отсортированный массив:
17 18 30 33 40 45 50 52
```

Рисунок 5 - Пример выполнения задачи №2

5.Контрольные вопросы

1. Что такое внутренняя сортировка и методы поиска информации?

– Внутренняя сортировка — это упорядочивание данных, которые полностью помещаются в оперативной памяти (ОЗУ) компьютера. Алгоритмы работают непосредственно с данными в памяти, что обеспечивает высокую скорость.

– Методы поиска информации — это алгоритмы для нахождения заданного элемента (или элементов) в структуре данных (например, в массиве, списке, графе). Цель — определить наличие элемента, его местоположение или вернуть сам элемент.

2. В чем заключается значение и цели сравнительного анализа алгоритмов сортировки?

Значение: Позволяет выбрать наиболее подходящий алгоритм для конкретной задачи, основываясь на объективных критериях, а не на интуиции.

Цели:

– Оценка эффективности: Сравнение алгоритмов по времени выполнения (временная сложность) и потреблению памяти (пространственная сложность).

– Определение применимости: Выявление сильных и слабых сторон алгоритмов в разных условиях (например, на частично отсортированных данных, на небольших или больших наборах).

– Теоретическое и практическое понимание: Анализ помогает глубже понять принципы работы алгоритмов и предсказать их поведение на реальных данных.

3. Какова сущность и применение основных алгоритмов внутренней сортировки (пузырьком, вставками, выбором)?

Сортировка пузырьком:

– Сущность: Последовательно сравниваются соседние элементы и меняются местами, если они стоят в неправильном порядке. Процесс повторяется, пока массив не будет отсортирован.

– Применение: В основном используется в учебных целях из-за простоты понимания. На практике не применяется для больших данных из-за низкой эффективности ($O(n^2)$).

Сортировка вставками:

– Сущность: Массив условно делится на отсортированную и неотсортированную части. Элементы из неотсортированной части по одному вставляются на правильную позицию в отсортированной части.

– Применение: Эффективен для небольших или почти отсортированных массивов ($O(n^2)$ в худшем случае, но $O(n)$ в лучшем). Широко используется в комбинации с другими алгоритмами (например, в Timsort).

Сортировка выбором:

– Сущность: Находится минимальный (или максимальный) элемент в неотсортированной части массива и меняется местами с первым элементом неотсортированной части. Процесс повторяется для оставшейся части.

– Применение: Прост в реализации, но неэффективен на больших данных ($O(n^2)$). Может быть полезен, когда стоимость записи в память высока, так как количество обменов минимально ($O(n)$).

4. Каковы особенности и эффективность основных алгоритмов поиска информации (бинарный, в глубину, в ширину)?

Бинарный поиск:

– Особенности: Работает *только на отсортированном массиве*. На каждом шаге интервал поиска уменьшается вдвое.

– Эффективность: Очень высокая. Временная сложность $O(\log n)$. Это стандартный выбор для поиска в отсортированных массивах.

Поиск в глубину (DFS):

– Особенности: Исследует граф, "углубляясь" в него по одному пути насколько возможно, прежде чем вернуться и пойти по другому пути. Использует стек (явно или через рекурсию).

– Эффективность: Временная сложность $O(V + E)$, где V — вершины, E — рёбра. Эффективен для проверки связности, поиска циклов, топологической сортировки.

Поиск в ширину (BFS):

– Особенности: Исследует граф "слоями", начиная от начальной вершины. Сначала посещает всех соседей, затем соседей соседей и т.д. Использует очередь.

– Эффективность: Временная сложность $O(V + E)$. Эффективен для поиска кратчайшего пути в невзвешенном графе и нахождения ближайших соседей.

5. Каковы особенности применения алгоритмов сортировки и поиска в различных сценариях программирования?

– Небольшие наборы данных: Простые алгоритмы (вставками, выбором) могут быть эффективнее сложных (быстрая сортировка, слиянием) из-за меньших накладных расходов.

– Большие наборы данных в памяти: Используются эффективные алгоритмы с временной сложностью O(n log n), такие как быстрая сортировка (Quicksort), сортировка слиянием (Mergesort) или пирамидальная сортировка (Heapsort).

– Данные, не помещающиеся в память: Применяется внешняя сортировка (например, многопутевое слияние), которая работает с данными на жестком диске.

– Поиск в статических данных: Данные сортируются один раз, после чего для поиска постоянно используется бинарный поиск.

– Поиск в динамических структурах: Используются структуры данных, совмещающие хранение и эффективный поиск (например, хеш-таблицы для константного времени O(1) или бинарные деревья поиска для времени O(log n)).

Поиск в графах:

– DFS применяется, когда нужно "исследовать" всю структуру (например, проход по директориям файловой системы).

– BFS применяется, когда важен кратчайший путь (например, маршрутизация в сетях, поиск соседей в социальных графах).